Egyedi felsorolók

Gregorics Tibor

gt@inf.elte.hu

http://people.inf.elte.hu/gt/oep

Felsorolások fajtái

- Eddig nevezetes gyűjtemények elemeinek szokásos, elvárt sorrendben történő, ún. standard felsorolásával találkoztunk. Ilyenkor többnyire nem jeleni meg önálló objektumként a felsorolást végző felsoroló, helyett egy indexváltozó, vagy egyéb segédváltozók jelzik a jelenlétét.
- Egyedi felsorolás az, amelyik vagy
  - egy nevezetes gyűjteményt nem standard módon sorol fel. Például:
    - mátrix elemeit nem sorfolytonosan járja be
    - nem az elejéről indul, hanem egy korábban (a First() művelettel) megkezdett felsorolást folytat
    - korábban leáll, mielőtt a gyűjtemény összes elemét bejárná
    - egy lépéshez a gyűjtemény több elemét is felhasználja
  - egyszerre több gyűjtemény szimultán bejárására épül
  - egy speciális, esetleg virtuális gyűjteményt jár be

1.Feladat

Adott két függvény, az $f : \mathbb{N} \longrightarrow \mathbb{R}$ és $g : \mathbb{N} \longrightarrow \mathbb{R}$, továbbá egy $e \in \mathbb{R}$ szám. Tudjuk, hogy van olyan i és j természetes számpár, amelyre f(i)+g(j)=e teljesül. Adjunk meg ilyen i-t és j-t!

$A = (e:\mathbb{R} , i:\mathbb{N} , j:\mathbb{N})$

$Ef = (e = e_0 \wedge \exists i,j \in \mathbb{N} : f(i)+g(j)=e)$

$Uf = (Ef \wedge f(i)+g(j)=e)$

Ez a specifikáció nem sok ötletet ad ahhoz, hogy melyik algoritmus mintát alkalmazzuk.

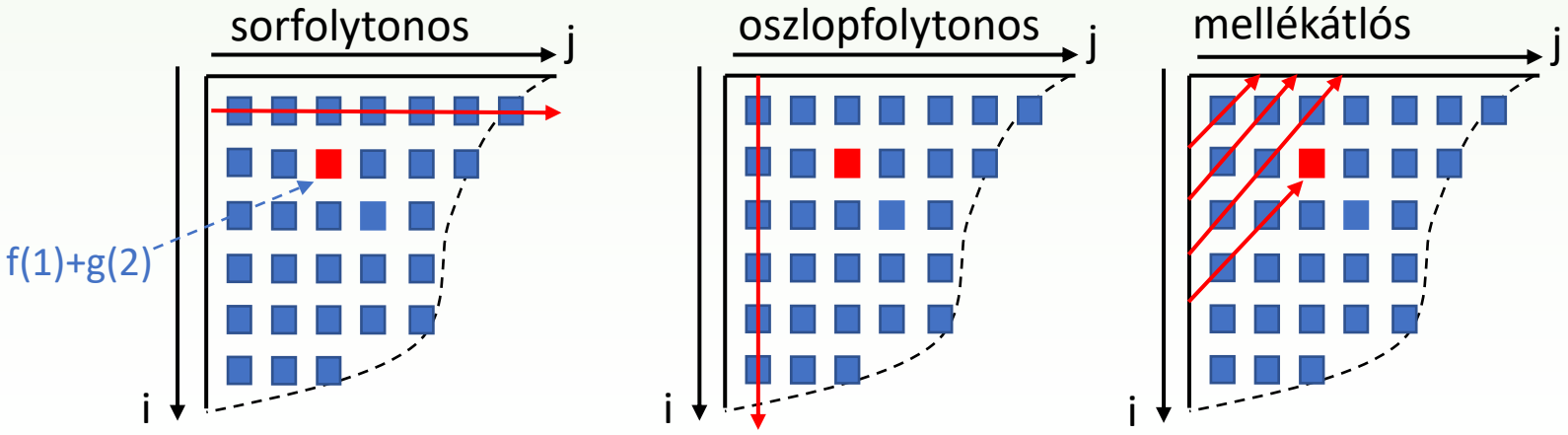
Ötlet

Gondolatban rendezzük el az f(i)+g(j) összegeket egy végtelen kiterjedésű 0-tól kezdve indexelt mátrixba úgy, hogy e mátrix i-dik sorának j-edik eleme az f(i)+g(j) érték legyen. Ebben a mátrixban kell megkeresnünk egy adott (e) értéket.

Definiáljunk egy felsorolót, amelyik ennek a képzeletbeli mátrix elemeinek indexpárjait járja be.

A standard (sorfolytonos vagy oszlopfolytonos) felsorolás nem lesz jó, mert a mátrix sorai is, oszlopai is végtelen hosszúak, így ezek a felsorolók csak az első sor vagy oszlop elemeit képesek bejárni.

De a mellékátlónkénti felsorolás segít.



Indexpárok felsorolója

A felsoroló mögött nem egy valódi, hanem egy virtuális gyűjtemény van.

Kell egy felsoroló, amely egy képzeletbeli végtelen kiterjedésű (0-tól kezdve indexelt) mátrix elemeinek sor-oszlop indexpárjait állítja elő.

$\mathrm{enor}(\mathbb{N}\times\mathbb{N})$

Számpárok végtelen sorozata. Az End() műveletet nem lehet definiálni, de szerencsére nem is lesz rá szükség.

$(\mathbb{N}\times\mathbb{N})^{\infty}$	First()	Next()	Current() : $\mathbb{N}\times\mathbb{N}$
i, j:$\mathbb{N}$	i, j:=0,0	**if** i>0 **then** i, j := i−1, j+1 **elsif** i=0 **then** i := j+1 ; j := 0	**return** (i, j)

Tervezés

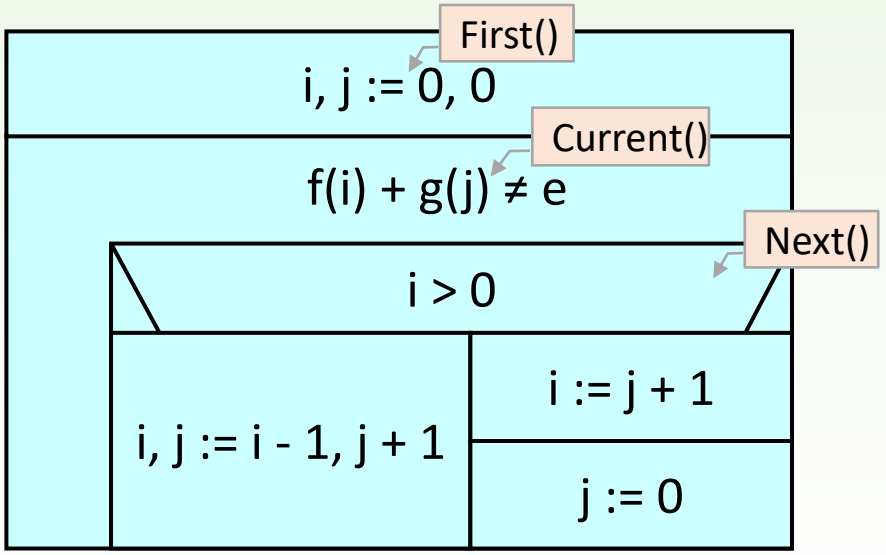
A = (t:enor($\mathbb{N}\times\mathbb{N}$), e:$\mathbb{R}$, i:$\mathbb{N}$, j:$\mathbb{N}$)

Ef = ($t = t_0 \wedge e = e_0 \wedge \exists i,j\in\mathbb{N}: f(i)+g(j)=e$)

Uf = ($e = e_0 \wedge (i,j) = \text{SELECT}_{(i,j)\in t_0}\ (f(i)+g(j)=e)$)

Kiválasztás:

t:enor(E)	~	t:enor($\mathbb{N}\times\mathbb{N}$)
$e\in t$	~	$(i,j)\in t$
elem	~	(i,j)
felt(e)	~	f(i)+g(j) = e



Kiválasztás tesztelése

- A kiválasztás tesztelése a felsoroló tesztelésére korlátozódik. Nem releváns azonban a felsorolás hossza, illetve utolsó eleme szerinti tesztelés. Ezek helyett azokat az eseteket érdemes vizsgálni, amikor a keresett elem
  - az első, második, illetve sokadik indexpárhoz tartozik
  - az első, második, illetve sokadik „mellékátlóba” esik
  - egy „mellékátló” elején, végén, vagy közepén helyezkedik el

Program

```
#include "read.hpp"

bool all(double r) { return true; }

int main()
{
        double e = read<double>("Give a real number: ",
                                "This is not a real number!", all);

        int i, j;
        i = j = 0;
        while( f(i)+g(j)!=e ){
                if(i>0) { --i; ++j; }
                else    { i = j+1; j = 0; }
        }

        cout << "The given number is equal to the sum f("
                << i << ")+g(" << j << ")\n";

        return 0;
}
```

a harmadik paraméter egy logikai függvény, amely eldönti, hogy a paraméterének adott érték megfelelő-e.

Ez egy függvénysablon, amelyet itt valós szám beolvasására használunk, ezért a típust helyettesítő paraméterének a double-t adjuk meg.

Olvasó függvénysablon

```
#include <iostream>

template <typename Item>
Item read( const std::string &msg, const std::string &err, bool valid(Item))
{
      Item n;
      bool wrong;
      do{
            std::cout << msg;
            std::cin >> n;
            if((wrong = std::cin.fail())) std::cin.clear();
            std::string tmp = "";
            getline(std::cin, tmp);
            wrong = wrong || tmp.size()!=0 || !valid(n);
            if(wrong) std::cout << err << std::endl;
      }while(wrong);
      return n;
}
```

Függvénysablon, amely definíciójának egyik típusa egy később megadandó paraméter.

függvényt váró paraméter

Az Item helyébe olyan típust lehet írni, amelyre értelmezett az operator>>()

read.hpp

2.Feladat

Egy kiránduláson adott távolságonként (lépésenként) mértük a felszín tengerszint feletti magasságát (méterben), és az adatokat egy szekvenciális inputfájlban rögzítettük.
A kirándulás hány százalékában vezetett az út felfelé?

$A = (f:infile(\mathbb{Z}), v:\mathbb{R})$

$Ef = (f = f_0 \wedge |f_0| \geq 2)$

$Uf = (v = (100 \cdot \sum_{i=2\,..\,|f_0|,\ f_0[i] > f_0[i-1]} 1) / (\sum_{i=2\,..\,|f_0|} 1))$

Itt a szekvenciális inputfájlnak egyszerre két egymás utáni elemére kell hivatkozni, amit a fájl standard felsorolása nem támogat.

$A = (t:enor(\mathbb{Z} \times \mathbb{Z}), v:\mathbb{R})$

$Ef = (t = t_0 \wedge |t_0| > 0)$

$Uf = (v = (100 \cdot \sum_{(első,második) \in t_0,\ első<második} 1) / (\sum_{(első,második) \in t_0} 1))$

Úgy kell a szekvenciális fájl elemeit felsorolni, hogy mindig annak két közvetlenül egymás utáni elemét tudjuk elérni.

Pufferelt felsoroló

A felsorolás minden lépésben a soron következő két szomszédos elemet olvassa be a szekvenciális inputfájlból.

$enor(\mathbb{Z}\times\mathbb{Z})$

Olyan sorozat, amelynek elemei az eredeti inputfájl szomszédos elemeiből álló számpárok.

$(\mathbb{Z}\times\mathbb{Z})^*$	First()	Next()	Current() : $\mathbb{Z}\times\mathbb{Z}$	End() : $\mathbb{L}$
f: infile($\mathbb{Z}$) első, másod : $\mathbb{Z}$ st : Status	st,első,f : read st,másod,f : read	első := másod st,másod,f : read	**return** (első, másod)	**return** st=abnorm

Tervezés

A = (t: enor($\mathbb{Z}\times\mathbb{Z}$), v:$\mathbb{R}$)

Ef = ($t = t_0 \wedge |t_0|>0$)

Uf = ($v = (100\cdot\sum_{(első,másod)\in t_0,\ első<másod} 1) / (\sum_{(első,másod)\in t_0} 1)$)

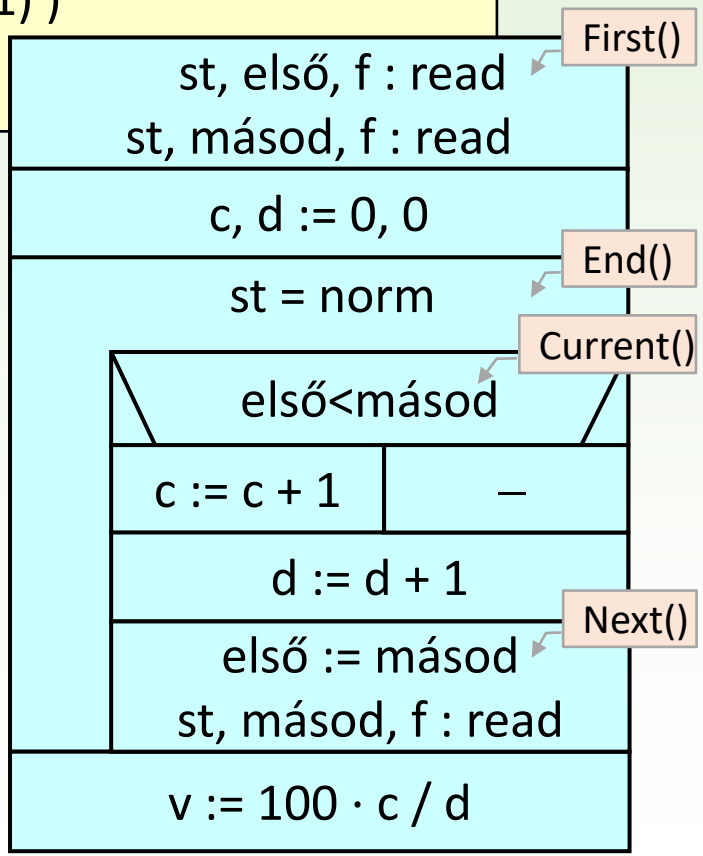
A számlálás és az összegzést közös ciklusba vonjuk össze.

Számlálás:

t:enor(E)	~	t:enor($\mathbb{Z}\times\mathbb{Z}$)
e∈t	~	(első, másod)∈t
felt(e)	~	első<másod

Összegzés:

t:enor(E)	~	t:enor($\mathbb{Z}\times\mathbb{Z}$)
e∈t	~	(első, másod)∈t
f(e)	~	1
H,+,0	~	$\mathbb{N}$,+,0



Számlálás és összegzés tesztelése

- A számlálás és összegzés ugyanazt a felsorolót használja, amelyet az alábbi szempontok szerint tesztelünk:
  - felsorolás hossza: egy, kettő vagy hosszabb (legyen végig emelkedő)
  - felsorolás eleje: csak az elején van egy emelkedés
  - felsorolás vége: csak a végén van egy emelkedés
- A számlálás eredménye szerinti tesztesetek:
  - nincs terep-emelkedés
  - egyetlen terep-emelkedés van
  - több terep-emelkedés van
- Az összegzés terheléses vizsgálata itt nem érdekes.

Program

```
int main()
{
	ifstream f("input.txt");
	if(f.fail()){
		cout << "Wrong file name!\n";
		return 1;
	}

	int first, second;
	int c = 0; int d = 0;
	for( f >> first >> second; !f.fail(); first = second, f >> second){
		if( first < second ) ++c;
		++d;
	}

	cout.setf(ios::fixed);
	cout.precision(2);
	cout << "Rate of the uphill part of the trip: " << (100.0*c)/d << "%" << endl;

	return 0;
}
```

```
f >> first >> second;
while( !f.fail() )
	if( first < second ) ++c;
	++d;
	first = second; f >> second;
}
```

valós osztás, mert a számlálóból double típusú érték lett

3.Feladat

Egy kiránduláson adott távolságonként (lépésenként) mértük a felszín tengerszint feletti magasságát, és az adatokat egy szekvenciális inputfájlban rögzítettük. Milyen hosszú volt a leghosszabb egybefüggően emelkedő szakasza a túrának?

A = (f:infile($\mathbb{Z}$), max:$\mathbb{N}$)

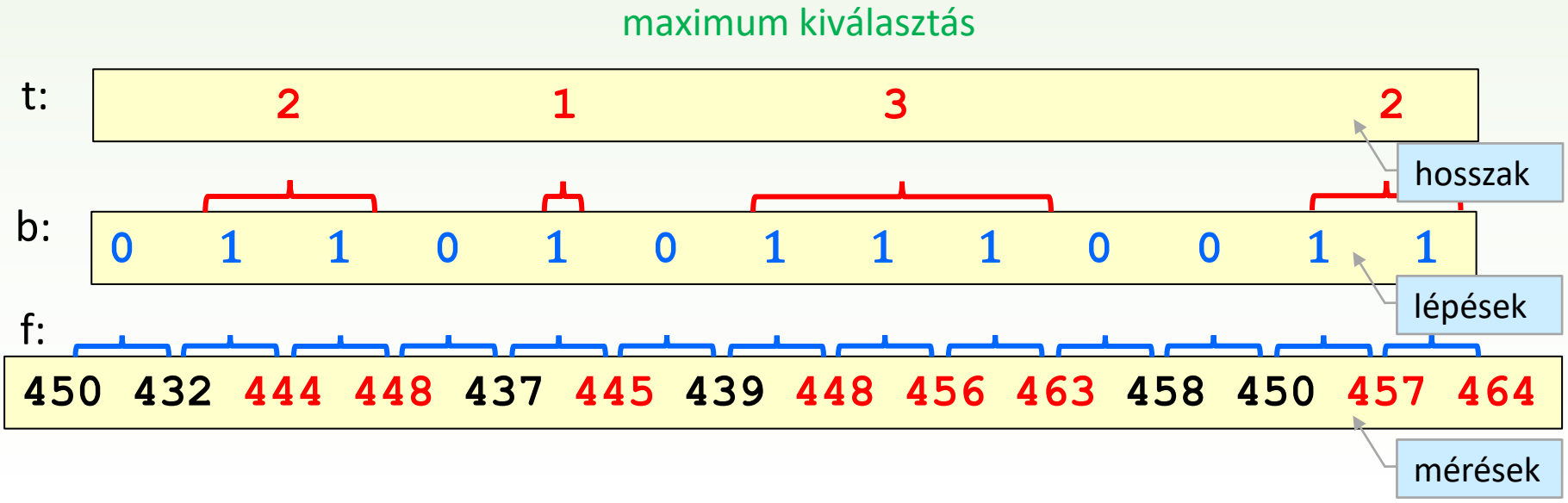
Ef = ($f = f_0 \wedge \exists i \in [1 .. |f|]: f[i] > f[i-1]$)

Uf = (MAX ?)

Egy maximum kiválasztásra lenne szükség, amely az egybefüggően emelkedő szakaszok hosszai között, de ezek a szakaszhosszok nem olvashatók ki közvetlenül az inputfájlból, ezért nehéz a feladatot specifikálni.

Absztrakciós szintek

- Könnyű lenne a feladat megoldása, ha az eredeti fájl adatai helyett a folyamatosan emelkedő szakaszok hosszait tudnánk felsorolni, mert ezen hosszok között egyszerű a legnagyobb értéket megtalálni.
- Az emelkedő szakaszok hosszainak megadásához viszont az eredeti fájl adatai helyett egyszerűbb volna csak annyit látni, hogy mely lépések emelkedtek, melyek nem.



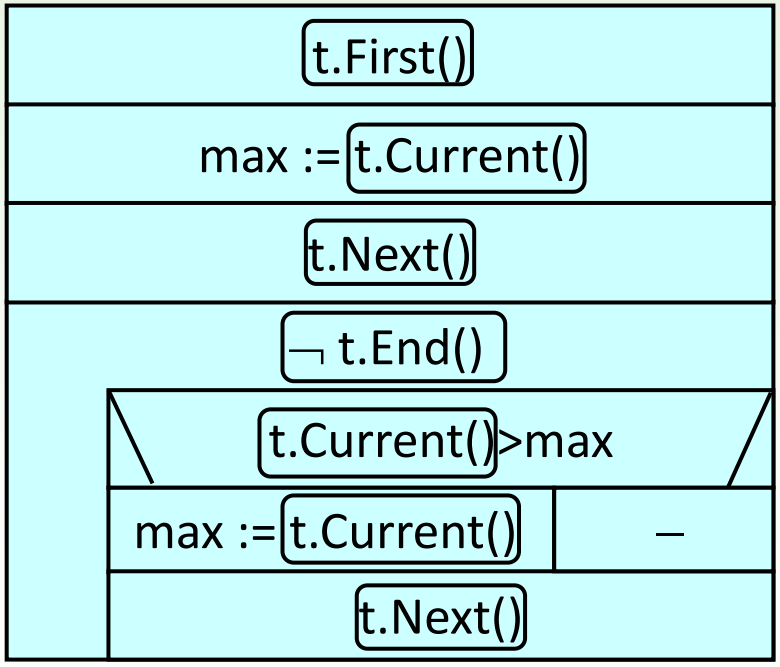
Tervezés

Tegyük fel, hogy van olyan felsorolónk, amely felsorolja az egybefüggően emelkedő szakaszok hosszait.

$A = (t:enor(\mathbb{N}), max:\mathbb{N})$

$Ef = (t = t_0 \wedge |t_0|>0)$

$Uf = (max = \mathbf{MAX}_{e \in t_0} e)$



Maximum kiválasztás

t:enor(E)	~	t:enor(ℕ)
f(e)	~	e
H, >	~	ℕ, >

Maximum kiválasztás tesztelése

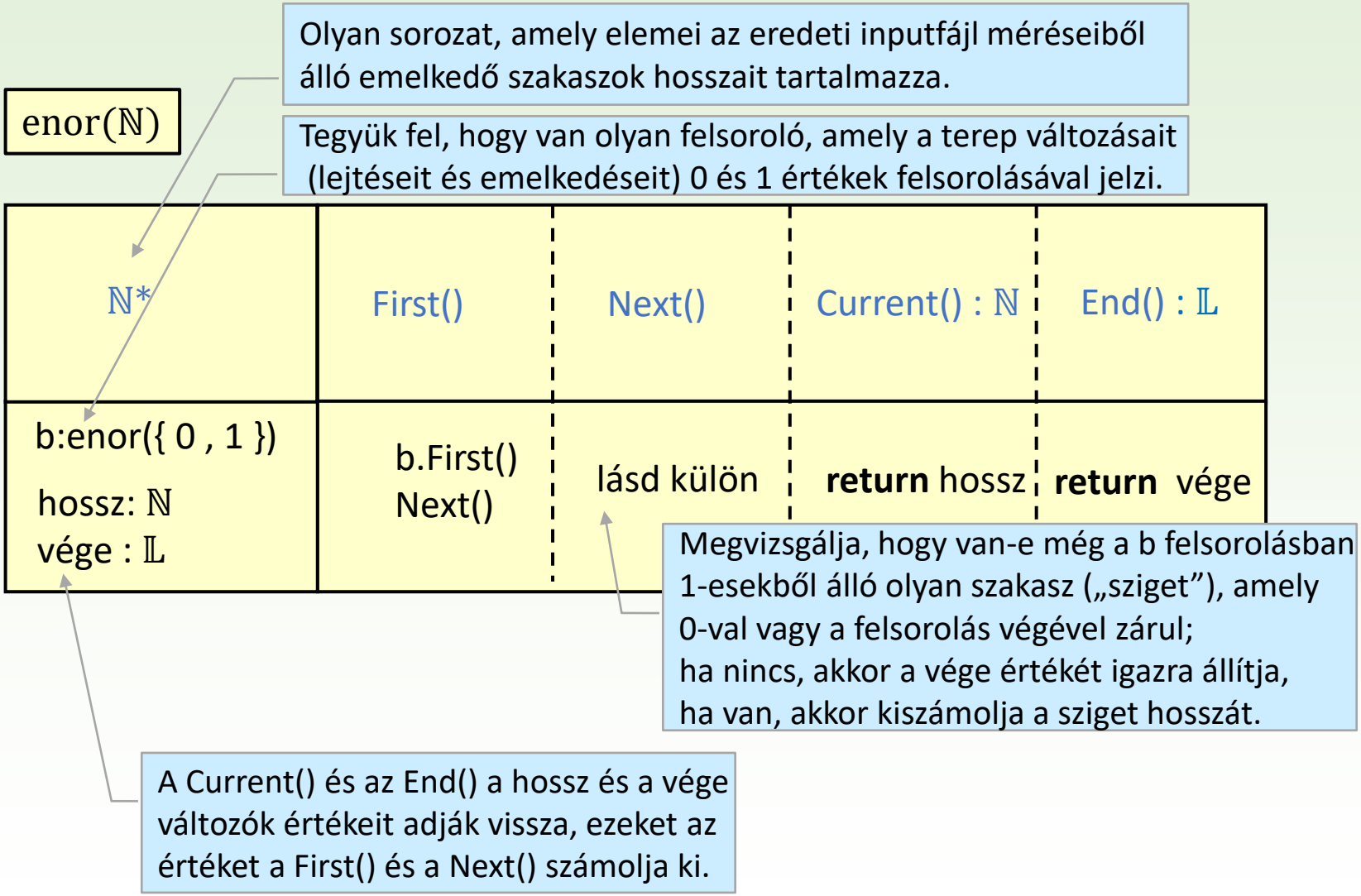
- Maximum kiválasztás felsorolója szerinti tesztesetek:
  - felsorolás hossza: egy, kettő vagy hosszabb
  - felsorolás eleje: legelső érték a legnagyobb
  - felsorolás vége: legutolsó érték a legnagyobb
- Maximum kiválasztás eredménye szerinti tesztesetek :
  - egyetlen legnagyobb érték
  - több egyformán legnagyobb érték

Főprogram

```
int main()
{
	try{
		LengthEnumerator t("input.txt");
		t.first();
		int max = t.current();
		for( t.next(); !t.end(); t.next() ){
			if( max < t.current() ) max = t.current();
		}
		cout << "The length of the longest uphill part: " << max << endl;
	} catch(FileOpenErrorException) {
		cout << "Wrong file name!\n" ;	}
	}
	return 0;
}
```

a megoldáshoz kitalált, a kirándulás emelkedő szakaszainak hosszát felsoroló objektum osztálya

Hosszak felsorolója



Next() művelet

b’ – a b változó kiinduló értéke (Ef-ben)
b” – a b változó értéke a 0-k átlépése után
a feldolgozás végén a b nem lesz feltétlenül üres

Megadja a b felsorolás soron következő 1-esekből álló, 0-val vagy a felsorolás végével határolt szakaszának hosszát, és hogy van-e ilyen.

A = (b:enor({ 0 , 1 }), hossz:$\mathbb{N}$, vége:$\mathbb{L}$)

a felsorolás korábban elkezdődött, még tart, utoljára érintett eleme e’

Ef = (b = b’ $\wedge$ b’ „folyamatban van” $\wedge$ e’=b.Current())

kiválasztja a következő 1-es elemet

Uf = ((l, e”, b”) = $\text{SEARCH}_{e \in (e', b')}$ (e=1) $\wedge$ (vége = $\neg$l) $\wedge$

először e’-t vizsgáljuk, majd b’-t kezdjük felsorolni

összegezi az aktuális csupa 1-esekből álló szakasz hosszát

addig tart, amíg $\neg$b.End() és b.Current()=1

$\wedge$ ($\neg$vége $\rightarrow$ ((hossz, b) = $\sum_{e \in (e'', b'')}^{e=1}$ 1)))

először e”-t vizsgáljuk, majd b”-t kezdjük felsorolni

(hossz, b) = $1 + \sum_{e \in b''} 1$

hiszen e” = 1

e – a felsorolásnál használt segédváltozó
e’ – e kezdőértéke, ami a b’ első (aktuális) eleme
e” – e értéke a keresés után, de az összegzés előtt

Next() művelet kicsit másképp

Megadja a b felsorolás soron következő 1-esekből álló, 0-val vagy a felsorolás végével határolt szakaszának hosszát, és hogy van-e ilyen.

A = (b:enor({ 0 , 1 }), hossz:$\mathbb{N}$, vége:$\mathbb{L}$)

Ef = (b = b' $\wedge$ b' „folyamatban van" $\wedge$ e'=b.Current())

keresi az első 1-es elemet vagy a felsorolás végét, ami biztos teljesül → kiválasztás

Uf = ((e", b") = $\text{SELECT}_{e \in (e', b')}$ (b.End() $\vee$ e=1) $\wedge$ (vége= (b"=<>)) $\wedge$

$\wedge$ ($\neg$vége $\rightarrow$ ((hossz, b) = $\sum_{e \in (e'', b'')}^{e=1} 1$)))

Specifikációs jelölések

Felsorolás végig:

t:enor(E)	i:[m .. n]	x:infile(E)
$r = \boxtimes_{e \in t_0} f(e)$	$r = \boxtimes_{i=m..n} f(i)$	$r = \boxtimes_{dx \in x_0} f(dx)$
$r, t = \boxtimes_{e \in t_0} felt(e)$	$r, i = \boxtimes_{i=m..n} felt(i)$	$r, (sx,dx,x) = \boxtimes_{dx \in x_0} felt(dx)$

Σ, MAX

SEARCH, SELECT

lehetne: $(sx,dx) \in x_0$

Az sx, dx a felsorolást támogató olyan segédváltozók, amelyekne kfeldolgozás végi értéke is hasznos lehet.

Felsorolás feltétel fenn állásáig:

t:enor(E)	i:[m .. n]	x:infile(E)
$\ldots = \boxtimes_{e \in t_0}^{fenáll(e)} f(e)$	$\ldots = \boxtimes_{i=m..n}^{fennáll(i)} f(i)$	$\ldots = \boxtimes_{dx \in x_0}^{fennáll(dx)} f(dx)$

$\neg t.End() \wedge fennáll(e)$

$i \leq n \wedge fennáll\ (i)$

$sx = norm \wedge fennáll(dx)$

Korábban abbahagyott felsorolás folytatása:

t:enor(E)	i:[m .. n]	x:infile(E)
$\ldots = \boxtimes_{e \in (t'.Current(), t')} f(e)$	$\ldots = \boxtimes_{i=i'+1..n} f(i)$	$\ldots = \boxtimes_{dx \in (dx', x')} f(dx)$

Next() művelet

$$e'', b'' = \mathrm{SELECT}_{e \in (e', b')}\ (b.\mathrm{End}() \vee e=1)$$

$$\wedge\ (\ \text{vége} = b''.\mathrm{End}()\ )$$

$$\wedge\ (\ \neg\text{vége} \rightarrow ((\text{hossz},b) = \sum_{e \in (e'', b'')}^{e=1} 1\ )))$$

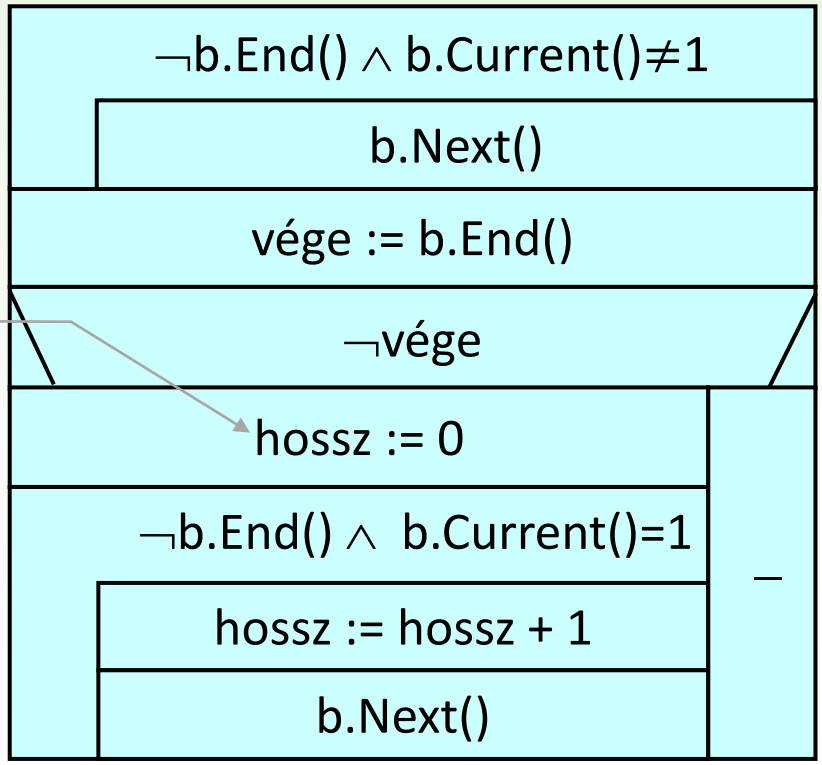
Kiválasztás megkezdett felsorolóval

t:enor(E)	~	b:enor({ 0 , 1 })
e∈t		First() nélkül
felt(e)	~	b.End() ∨ b.Current()=1

Feltételig tartó összegzés megkezdett felsorolóval

hossz := 1
b.Next()

t:enor(E)	~	b:enor({ 0 , 1 })
e∈t		First() nélkül amíg b.Current()=1
s	~	hossz
f(e)	~	1
H,+,0	~	ℕ,+,0



Next() szürkedoboz tesztelése

- Kiválasztás tesztesetei:
  - felsorolás hossza szerint: a fájl elején nulla, egy, vagy hosszabb nem emelkedő szakasz van
  - felsorolás eleje szerint: a fájl elején elején rögtön van emelkedés
  - felsorolás vége szerint: csak a fájl legvégén van egy lépéses emelkedés
  - A kiválasztás feltétele szerint: a fájlban nincs emelkedés illetve van

- Feltételig tartó összegzés tesztesetei:
  - felsorolás hossza szerint : a fájl elején nulla, egy, vagy hosszabb emelkedő szakasz van
  - felsorolás eleje szerint : csak a fájl elején van egy emelkedés
  - felsorolás vége szerint : csak a fájl végén van egy lépéses emelkedés
  - Az összegzés skálázása: itt nem érdekes

Emelkedések hosszainak felsorolója

```
class LengthEnumerator{
public:
      LengthEnumerator(const std::string &fname) : _b(fname){}
      void first() { _b.first(); next(); }
      int  current()  const { return _length; }
      bool end()      const { return _end; }
      void next();
private:
      BitEnumerator _b;
      int _length;
      bool _end;
};
```

a megoldáshoz kitalált, a kirándulás emelkedő lépéseit 1-gyel, egyéb lépéseit 0-val helyettesítő felsorolás objektumának osztálya

```
void LengthEnumerator::next()
{
      for( ; !_b.end() && _b.current()==0; _b.next() );
      if ( (_end = _b.end()) ) return;
      for( _length = 0 ; !_b.end() && _b.current()==1; _b.next() ) ++_length;
}
```

Lépések felsorolója

enor({ 0 , 1 })

Olyan sorozat, amely a kirándulás emelkedő lépéseit 1-gyel, egyéb lépéseit 0-val helyettesíti. A felsorolás hossza az eredeti inputfájl hosszánál eggyel kisebb (hiszen minden eleme az eredeti fájl két szomszédos mérésének felel meg.

{0, 1}*	First()	Next()	Current() : {0, 1}	End() : 𝕃
f: infile(ℤ) első, másod:ℤ st:Status	st, első, f : read st, másod, f : read	első:= másod st, másod, f : read	**if** első < másod **then return** 1 **else return** 0	**return** st=abnorm

Lépések felsoroló osztálya

```
class FileOpenErrorException : std::exception {};

class BitEnumerator{
public:
	BitEnumerator(const std::string &fname){
		_f.open(fname);
		if(_f.fail()) throw new FileOpenErrorException;
	}
	void first() { _f >> _first >> _second; }
	void next()  { _first = _second; _f >> _second; }
	int  current() const { return (_first < _second ? 1 : 0); }
	bool end()     const { return _f.fail(); }
private:
	std::ifstream _f;
	int _first, _second;
};
```

feltételes kifejezés:
ha _first < _second fenn áll.
akkor az értéke 1, különben 0